**UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA**

**CAMPUS JUTIAPA**

**FACULTAD DE INGENIERIA EN SISTEMAS Y CIENCIA DE LA COMPUTACION**

# MIGRACION LISTAS ENLAZADAS A JAVA SCRIPT

**TANIA ANDREA MIRANDA RAMIREZ**

**0905-24-16058**

**PROGRAMACION III**

# Retos Lista Simple

```
PS C:\Users\Tara Lynn> node -v
v22.19.0
PS C:\Users\Tara Lynn> npm -v
10.9.3
```

```
countOccurrences(value) {
  throw new Error(
    "TODO RETO: Implementar countOccurrences(value) en SinglyLinkedList."
  );
}
```

```
countOccurrences(value) {
  let count = 0;
  let current = this.head;

  while (current !== null) {
    if(current.value === value) {
      count++;
    }
    current = current.next;
  }
```

```
=== RETOS NODE.JS (PENDIENTES) ===
Retos Lista Simple
Datos base: [A, B, A, C, B, D]
  [OK] countOccurrences('A') esperado=2 -> 2
```

```
clean() {
  throw new Error("TODO RETO: Implementar clean() en SinglyLinkedList.");
}
```

```
clean() {

  this.head = null;
  this.tail = null;
  this._size = 0;

}
```

```
=== RETOS NODE.JS (PENDIENTES) ===
Retos Lista Simple
Datos base: [A, B, A, C, B, D]
  [OK] countOccurrences('A') esperado=2 -> 0
  [OK] clean() esperado=6 -> undefined
```

```
  reverseInPlace() {
    throw new Error(
      "TODO RETO: Implementar reverseInPlace() en SinglyLinkedList."
    );
  }
```

```
  reverseInPlace() {
    let previos = null;
    let current = this.head;
    let next = null;

    while (current !== null) {
      next = current.next;
      current.next = previos;
      previos = current;
      current = next;
    }

    this.head = previos;

  }
```

```
=== RETOS NODE.JS (PENDIENTES) ===
Retos Lista Simple
Datos base: [A, B, A, C, B, D]
  [OK] countOccurrences('A') esperado=2 -> 0
  [OK] clean() esperado=6 -> undefined
  [OK] reverseInPlace() esperado=[6, 5, 4, 2] -> [6, 5, 4, 2]
```

```
  removeDuplicates() {
    throw new Error(
      "TODO RETO: Implementar removeDuplicates() en SinglyLinkedList."
    );
  }
```

```
  removeDuplicates() {
    let current = this.head;

    while(current !== null && current.next !== null) {
      let runner = current;

      while(runner.next !== null) {
        if(runner.next.value === current.value) {
          runner.next = runner.next.next;
        } else {
          runner = runner.next;
        }
      }

      current = current.next;
    }
  }
```

```
=== RETOS NODE.JS (PENDIENTES) ===
Retos Lista Simple
Datos base: [A, B, A, C, B, D]
  [OK] countOccurrences('A') esperado=2 -> 0
  [OK] clean() esperado=6 -> undefined
  [OK] reverseInPlace() esperado=[6, 5, 4, 2] -> [6, 5, 4, 2]
  [OK] removeDuplicates() en [A, B, A, C, B, D] -> undefined
```

# Retos Lista Doble

```
countOccurrences(value) {
  throw new Error(
    "TODO RETO: Implementar countOccurrences(value) en DoublyLinkedList."
  );
}
```

```
countOccurrences(value) {
  let count = 0;
  let current = this.head;

  while (current !== null) {
    if(current.value === value) {
      count++;
    }
    current = current.next;
  }
  return count;
}
```

```
Retos Lista Doble
Datos base: [v1, v2, v3, v2, v4]
  [OK] countOccurrences('v2') esperado=2 -> 2
```

```
clean() {
  throw new Error("TODO RETO: Implementar clean() en DoublyLinkedList.");
}
```

```
clean() {
  this.head = null;
  this.tail = null;
  this._size = 0;
}
```

```
Retos Lista Doble
Datos base: [v1, v2, v3, v2, v4]
  [OK] countOccurrences('v2') esperado=2 -> 2
  [OK] clean() esperado=5 -> undefined
```

```
reverseInPlace() {
  throw new Error(
    "TODO RETO: Implementar reverseInPlace() en DoublyLinkedList."
  );
}
```

```javascript
reverseInPlace() {
  let current = this.head;
  let temp = null;

  while (current !== null) {
    temp = current.previous;
    current.previous = current.next;
    current.next = temp;
    current = current.previous;
  }

  if (temp !== null) {
    temp = this.head;
    this.head = this.tail;
    this.tail = temp;
  }
}
```

```
Retos Lista Doble
Datos base: [v1, v2, v3, v2, v4]
  [OK] countOccurrences('v2') esperado=2 -> 2
  [OK] clean() esperado=5 -> undefined
  [OK] reverseInPlace() esperado=[40, 30, 20, 10] -> [40, 30, 20, 10]
```

```javascript
removeDuplicates() {
  throw new Error(
    "TODO RETO: Implementar removeDuplicates() en DoublyLinkedList."
  );
}
```

```javascript
removeDuplicates() {
  let current = this.head;

  while (current !== null && current.next !== null) {
    let runner = current;

    while(runner.next !== null) {
      if(runner.next.value === current.value) {
        let nodeToRemove = runner.next;
        runner.next = nodeToRemove.next;

        if (nodeToRemove.next !== null) {
          nodeToRemove.next.previous = runner;
        }

        if (nodeToRemove === this.tail) {
          this.tail = runner;
        }
      } else {
        runner = runner.next;
      }
    }

    current = current.next;

  }
}
```

```
Retos Lista Doble
Datos base: [v1, v2, v3, v2, v4]
  [OK] countOccurrences('v2') esperado=2 -> 2
  [OK] clean() esperado=5 -> undefined
  [OK] reverseInPlace() esperado=[40, 30, 20, 10] -> [40, 30, 20, 10]
  [OK] removeDuplicates() -> undefined
```